

Exploración de conceptos de conversión A/D con Raspberry Pi Pico 2W

Riveros Sierra Harol Felipe 1401660, Bohorquez Blanco Salome 1401654

Docente: Ing. José de Jesús Rugeles Uribe.

I. RESUMEN

Este informe presenta el desarrollo de una práctica de laboratorio orientada al estudio de la conversión analógico-digital mediante una Raspberry Pi Pico 2 W y el microcontrolador RP2350. Inicialmente, se analizó experimentalmente el muestreo de una señal senoidal, determinando la frecuencia de muestreo, la frecuencia de entrada y el número de muestras por período. Posteriormente, se evaluó el comportamiento estadístico del ADC ante diferentes niveles de tensión DC mediante 10 000 mediciones, analizando la media, la desviación estándar y la distribución de las lecturas. Finalmente, se estudió la resolución de 12 bits del ADC y su representación mediante la función `read_u16()` de MicroPython. Estos resultados permitieron analizar experimentalmente el muestreo, la cuantización, la variabilidad, la repetibilidad y la exactitud del sistema de adquisición.

II. ABSTRACT

This report details a laboratory exercise focused on the study of analog-to-digital conversion using a Raspberry Pi Pico 2 W and the RP2350 microcontroller. Initially, the sampling of a sinusoidal signal was analyzed experimentally by determining the sampling frequency, the input frequency, and the number of samples per period. Subsequently, the statistical behavior of the ADC was evaluated at various direct current (DC) voltage levels using 10,000 measurements, analyzing the mean, standard deviation, and distribution of the readings. Finally, the ADC's 12-bit resolution and its representation via the MicroPython `read_u16()` function were examined. These results enabled an experimental analysis of the acquisition system's sampling, quantization, variability, repeatability, and accuracy.

III. PALABRAS CLAVE

ADC, Raspberry Pi Pico 2W, muestreo, conversión analógico digital, cuantización, MicroPython, análisis estadístico.

IV. KEYWORDS

ADC, Raspberry Pi Pico 2W, sampling, analog-to-digital conversion, quantization, MicroPython, statistical analysis.

V. INTRODUCCIÓN

La conversión analógico-digital es un proceso fundamental en los sistemas de adquisición y procesamiento de señales, ya que permite representar una señal analógica mediante valores digitales que pueden ser procesados por un sistema computacional. En esta práctica se estudia experimentalmente este proceso utilizando una Raspberry Pi Pico 2 W, cuyo microcontrolador RP2350 dispone de un convertidor analógico-digital (ADC) con una resolución nominal de 12 bits.

El desarrollo de la práctica se divide en dos etapas principales. En la primera se analiza el proceso de muestreo de una señal senoidal, utilizando el osciloscopio para observar tanto la señal analógica como los instantes en los que el ADC realiza cada adquisición. A partir de estas mediciones se determina la frecuencia de muestreo, la frecuencia de la señal de entrada y el número de muestras obtenidas por período.

En la segunda etapa se analiza estadísticamente el comportamiento del ADC ante diferentes niveles de tensión DC. Para ello se realizan 10 000 mediciones por ensayo y se estudian parámetros como la media, la desviación estándar y la distribución de las lecturas. Además, se analiza la relación entre los códigos digitales generados por el ADC y los valores de tensión obtenidos mediante MicroPython, de esta manera, la práctica permite relacionar los conceptos teóricos de muestreo, cuantización y resolución con mediciones experimentales, evaluando el comportamiento y la variabilidad de un sistema de adquisición analógico-digital.

VI. OBJETIVOS

- Analizar experimentalmente el proceso de conversión analógico-digital mediante el ADC de la Raspberry Pi Pico 2 W.
- Determinar y comparar experimentalmente el período y la frecuencia de muestreo, la frecuencia de la señal de entrada y el número de muestras obtenidas por período.
- Caracterizar estadísticamente el comportamiento del ADC ante diferentes niveles de tensión DC mediante el análisis de la media, la desviación estándar, los histogramas y la variación de los códigos digitales obtenidos.
- Relacionar la resolución nominal de 12 bits del ADC con los niveles de cuantización, el tamaño ideal del LSB y la representación de las mediciones mediante la función `read_u16()` de MicroPython, considerando la exactitud y repetibilidad del sistema de adquisición.

VII. MARCO TEÓRICO

A. ¿Qué es un conversor analógico – digital?

Un conversor analógico-digital, también conocido como ADC (Analog-to-Digital Converter), es un dispositivo electrónico encargado de transformar una señal analógica, como un voltaje, en un valor digital que pueda ser procesado por un microcontrolador o sistema digital.

Las señales analógicas pueden tomar una cantidad continua de valores dentro de un determinado rango de voltaje. Por el contrario, los sistemas digitales trabajan con valores discretos representados mediante códigos binarios. El ADC permite realizar la conexión entre estos dos tipos de señales mediante un proceso de conversión.

De manera general, la conversión analógico-digital involucra el muestreo de la señal y la cuantización de su amplitud. Durante el muestreo se toman valores de la señal en determinados instantes de tiempo, mientras que durante la cuantización cada valor medido es asociado con uno de los niveles digitales disponibles.

En la guía de laboratorio, este proceso se analiza experimentalmente utilizando la Raspberry Pi Pico 2 W y su convertidor A/D, observando tanto los instantes de muestreo como los valores digitales obtenidos durante la adquisición de una señal.

B. Rango de cuantización

El rango de cuantización corresponde al intervalo de valores analógicos que puede representar un convertidor analógico-digital mediante códigos digitales.

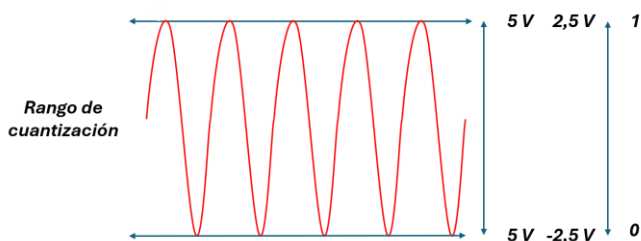


Figura 1. Representación del rango de cuantización para una señal analógica

Para comprender el concepto de rango de cuantización, primero es necesario conocer los niveles de cuantización. Estos niveles corresponden a las divisiones o valores discretos en los que se divide un rango determinado de voltaje analógico. Cada nivel representa una posible salida digital del convertidor analógico-digital.

Para tomar un ejemplo sencillo, tenemos un ADC de 3 bits, el número de cuantización sería:

$$L = 2^N$$

Donde N corresponde al número de bits del convertidor, por lo tanto:

$$L = 2^3 = 8$$

Esto significa que el rango de voltaje de entrada se divide en 8 niveles de cuantización, si por ejemplo el rango del ADC es de 0 a 5V, cada una de estas divisiones representa aproximadamente una pequeña porción del rango total, en la siguiente figura se muestra cómo se vería este nivel de cuantización:

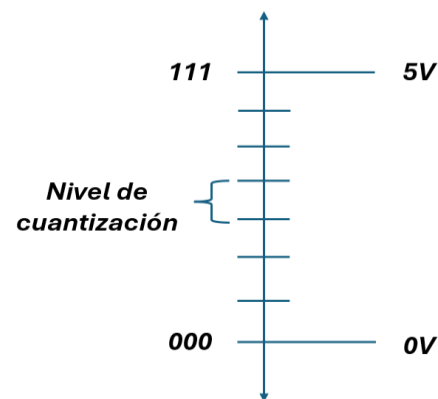


Figura 2. Representación del nivel de cuantización

El tamaño de cada división está determinado por:

$$\Delta = \frac{\text{Rango de cuantización}}{2^N}$$

Tomando el ejemplo de los 3 bits:

$$\Delta = \frac{5V}{2^3} = 625mV$$

Por lo tanto, los niveles de cuantización son las divisiones disponibles dentro del rango de entrada, mientras que el rango de cuantización es el intervalo total de valores analógicos que el ADC puede convertir.

C. INL VS DIGITAL CODE EXAMPLE

La INL (*Integral Non-Linearity* o no linealidad integral) es un parámetro utilizado para evaluar qué tan cerca se encuentra el comportamiento real de un ADC respecto a su comportamiento ideal. Idealmente, existe una relación lineal entre el voltaje de entrada y el código digital generado por el ADC. Esto significa que, al aumentar progresivamente el voltaje de entrada, el código digital debería aumentar de manera uniforme.

La INL representa la desviación acumulada entre la respuesta real del ADC y la respuesta ideal esperada. Generalmente, este error se expresa en unidades de LSB.

Un ejemplo conceptual puede observarse mediante la relación:

$$V_{in} \rightarrow \text{Código digital}$$

D. ¿Qué es un LSB y MSB?

LSB significa Least Significant Bit o bit menos significativo, en un ADC, el LSB representa el cambio mínimo ideal en el voltaje de entrada que puede producir un cambio de un código digital al siguiente.

El tamaño de in LSB depende principalmente de:

- El voltaje de referencia
- La resolución del ADC

Para un ADC de N bits:

$$LSB = \frac{V_{REF}}{2^N}$$

Para el ADC de 12 bits y un voltaje de referencia de 3.3V, se obtiene:

$$LSB = \frac{3.3V}{2^{12}} = 0.806mV$$

Esto significa que, idealmente, cada incremento de un código digital representa aproximadamente 0.806mV.

Por ejemplo, el cambio entre los códigos :

$$2048 \rightarrow 2049$$

Representaría aproximadamente un incremento de un LSB en el voltaje de entrada.

Por otro lado, el MSB (*Most Significant Bit*) corresponde al bit de mayor peso dentro del código digital. En un ADC de 12 bits, este corresponde al bit 2^{N-1} , Considerando una referencia de 3.3V, el MSB representa una contribución de aproximadamente 1.65V, calculada como:

$$MSB = \frac{2^{N-1} \times V_{REF}}{2^N}$$

Por lo tanto, el MSB representa la mayor contribución individual dentro de la conversión digital, mientras que los demás bits poseen pesos progresivamente menores.

E. Error RMS mínimo de cuantización

Durante el proceso de cuantización, un ADC debe asignar un valor analógico continuo a uno de los niveles digitales disponibles. Como consecuencia, generalmente existe una pequeña diferencia entre el valor analógico real y el valor representado digitalmente, esta diferencia se conoce como error de cuantización.

Si se considera una distribución uniforme del error de cuantización su valor eficaz o RMS esta dado por:

$$ERROR_{RMS} = \frac{LSB}{\sqrt{12}}$$

Para un ADC de 12 bits y un voltaje de referencia de 3.3V

$$LSB = 0.806mV$$

$$ERROR_{RMS} = \frac{0.806mV}{\sqrt{12}} = 0.233mV$$

Este valor representa el nivel RMS mínimo de ruido asociado idealmente al proceso de cuantización.

En condiciones reales, las mediciones pueden presentar variaciones adicionales producidas por ruido eléctrico, interferencias y otras características del sistema de adquisición.

F. SNR teórico máximo de un ADC

La relación señal a ruido, conocida como SNR (Signal-to-Noise Ratio), permite comparar la magnitud de la señal útil con el ruido presente en un sistema.

En un ADC ideal, el ruido generado por la cuantización establece un límite teórico para la relación señal a ruido.

Para una señal senoidal que utiliza adecuadamente el rango completo del ADC, el SNR teórico máximo puede aproximarse mediante:

$$SNR_{m\acute{a}x} = 6.02 N + 1.76 dB$$

Donde N es el numero de bits del ADC, tomando de ejemplo del ADC de 12 bits, obtendríamos:

$$SNR_{m\acute{a}x} = 6.02 (12) + 1.76 dB$$

$$SNR_{m\acute{a}x} = 74 dB$$

Este resultado corresponde a una condición ideal, considerando que el principal ruido presente es el producido por la cuantización. En un sistema real, el SNR puede ser menor debido a fuentes adicionales de ruido.

G. Raspberry Pi Pico 2W

La Raspberry Pi Pico 2W es una tarjeta de desarrollo de bajo costo basada en el microcontrolador RP2350, de arquitectura ARM Cortex-M33 de doble núcleo, que incorpora además un módulo de conectividad inalámbrica Wi-Fi/Bluetooth. La tarjeta expone en sus pines de propósito general (GPIO) varios periféricos de comunicación, entre ellos hasta dos módulos UART, dos buses I2C y dos buses SPI, los cuales pueden asignarse a distintos pines físicos según la necesidad del proyecto. Para esta práctica de laboratorio la Raspberry Pi Pico 2W es utilizada como sistema de adquisición, aprovechando su convertidor analógico-digital integrado para realizar mediciones de señales eléctricas.

En la siguiente figura se muestra la ubicación de los respectivos pines en La Raspberry Pi Pico 2W:

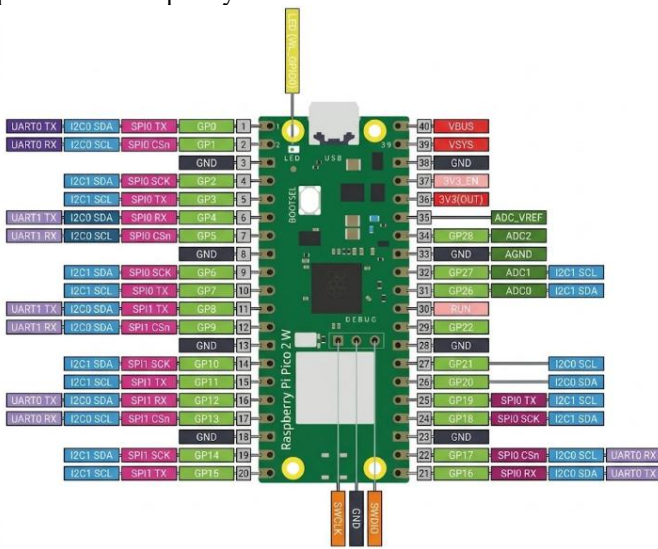


Figura 3. Ubicación de los pines en La Raspberry Pi Pico 2W.

VIII. METODOLOGÍA/PROCEDIMIENTO EXPERIMENTAL

A. Observación experimental del proceso de muestreo

En la parte I del laboratorio se pide que la señal analógica aplicada ingrese por el pin GP26 a partir de esto cada vez que el programa realice una lectura en el ADC, este genere un pulso digital en el GP15.

De esta forma, utilizando dos canales del osciloscopio será posible observar simultáneamente:

- CH1: señal analógica que está siendo muestreada;
- CH2: pulsos que indican los instantes de muestreo.

En la siguiente figura se muestra la ubicación de los respectivos pines en La Raspberry Pi Pico 2W:

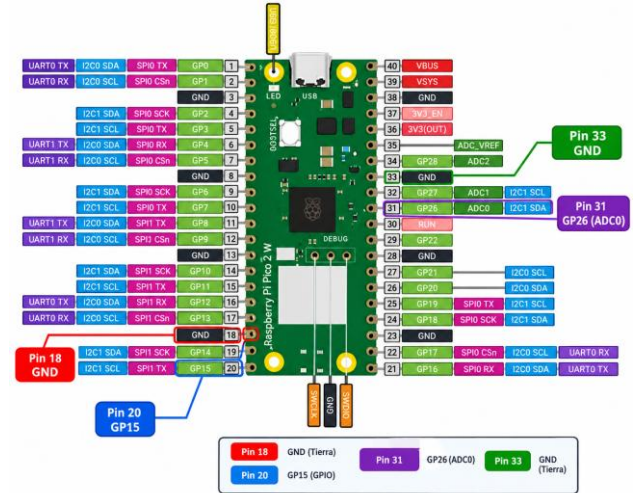


Figura 4. Pines utilizados en la parte I

Cabe resaltar que el ADC del RP2350 tiene una resolución nominal de 12 bits lo que permite obtener 4096 niveles de cuantización y códigos digitales desde 0 hasta 4095, sin embargo, dentro del MicroPython se utiliza la función `read_u16()` la cual entrega valores entre:

$$0 \leq \text{raw16} \leq 65535$$

Al utilizar `read_u16()` no está alterando los niveles de cuantización de los 12 bits, lo único que hace es cambiar la escala de medición, tomando un ejemplo coloquial, tenemos una escala de 0 a 10, pero alguien decide expresarla de 0 a 100, visto de otra forma:

$$\frac{5}{10} \approx \frac{50}{100}$$

Es exactamente la misma medición solo que en escalas diferentes.

Teniendo lo anterior claro, la conexión de la Raspberry con el osciloscopio se vería de la siguiente manera:

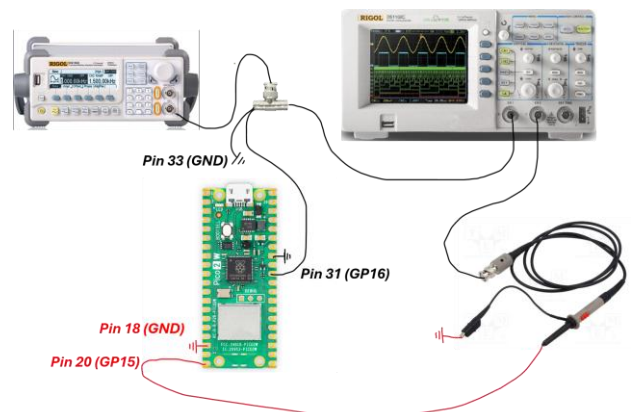


Figura 5. Montaje de la parte I del laboratorio[1]

Al realizar el montaje, se procedió a verificar que la señal seleccionada se encontrara dentro del rango permitido por la Raspberry Pi Pico 2 W, evitando valores negativos o superiores a 3,3 V. Una vez realizada esta comprobación, se conectó la señal senoidal al pin GP26 de la Raspberry Pi Pico 2 W y al canal 1 del osciloscopio. Posteriormente, se ejecutó el código `sampling_1.py` en MicroPython, el cual permite generar pulsos digitales en el pin GP15 cada vez que se realiza una lectura del ADC. De esta manera, mediante el osciloscopio fue posible observar simultáneamente la señal analógica y los flancos ascendentes correspondientes aproximadamente a los instantes de muestreo.

A partir de la señal observada en el canal 2 del osciloscopio, se realizaron mediciones utilizando los cursores para determinar la diferencia de tiempo entre varios flancos ascendentes consecutivos. Con estos valores se obtuvo experimentalmente el período y la frecuencia de muestreo, los cuales posteriormente fueron comparados con los valores calculados por el software.

Posteriormente, se utilizó el canal 1 del osciloscopio para medir el período de la señal senoidal y calcular su frecuencia real. Con la frecuencia de muestreo y la frecuencia de entrada obtenidas experimentalmente, se determinó el número de muestras disponibles por cada período de la señal.

Para observar el efecto de la frecuencia de entrada sobre el muestreo, se mantuvo la frecuencia de muestreo en 1000 Hz y se modificó la frecuencia del generador a 100, 200, 250 y 400 Hz. Para cada valor se midió la frecuencia real mediante el osciloscopio y se calculó nuevamente el número de muestras por período.

B. Análisis estadístico

Para la segunda parte de la práctica, la entrada GP26/ADC0 de la Raspberry Pi Pico 2 W donde su ubicación se evidencia en la figura 4, se conecta a una fuente de tensión DC. La tensión aplicada debe mantenerse dentro del rango permitido de 0 a 3,3 V. Esta conexión permite al ADC realizar múltiples mediciones de una tensión aproximadamente constante para posteriormente analizar estadísticamente los valores obtenidos.

En la siguiente figura se muestra las conexiones para la segunda parte del laboratorio:

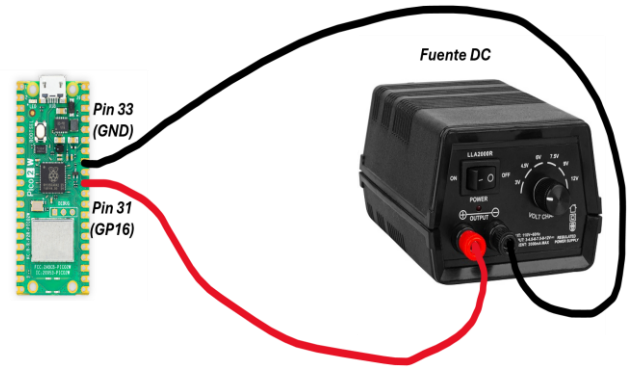


Figura 6. Montaje de la parte II del laboratorio.[1]

Realizado el montaje, para cada nivel de tensión seleccionado, se ejecutó el programa `sampling_2.py` en MicroPython, el cual realizó un total de 10 000 mediciones por ensayo. A partir de estas mediciones se obtuvieron los valores promedio y la desviación estándar de las lecturas entregadas por el ADC. Además, el programa generó los archivos correspondientes con las muestras obtenidas y la distribución de los códigos nominales del ADC.

Finalmente, los archivos generados fueron procesados mediante MATLAB para calcular la media y la desviación estándar, comparándolas con los valores obtenidos directamente por el programa. También se elaboraron gráficas de las mediciones en función del número de muestra e histogramas de los valores de tensión y de los códigos digitales. Con estos resultados se analizó la dispersión de las mediciones, la cantidad de códigos ADC obtenidos para una tensión aproximadamente constante, la variación de la desviación estándar con el nivel de tensión y la diferencia entre el valor promedio obtenido por el ADC y el valor medido con el multímetro.

IX. RESULTADOS

A. PREPARACIÓN DE LA SEÑAL DE ENTRADA (PARTE I)

Antes de ingresar cualquier señal a la Raspberry Pi Pico 2 W, se procedió a verificar que sus niveles de tensión fueran adecuados para evitar posibles daños en la entrada del dispositivo. Para ello, se estableció como condición que la señal $v(t)$ se encontrara dentro de un rango de 0 V a 3,3 V como máximo.

la señal seleccionada fue la siguiente:

$$v(t) = 1.65V + 2V \text{ SEN}(2\pi 100\text{HZ } t)$$

Hallamos V_p :

$$V_p = \frac{V_{pp}}{2} = \frac{2V_{pp}}{2} = 1V$$

Hallamos $V_{m\acute{a}x}$:

$$V_{m\acute{a}x} = K + V_p = 1.65V + 1V = 2.65V$$

Hallamos $V_{m\acute{i}n}$:

$$V_{m\acute{a}x} = K + V_p = 1.65V - 1V = 0.65V$$

Teóricamente el $V_{m\acute{a}x}$ y el $V_{m\acute{i}n}$ cumplen con el rango de especificado de 0 a 3.3V, a continuación se comprobará experimentalmente utilizando el osciloscopio:

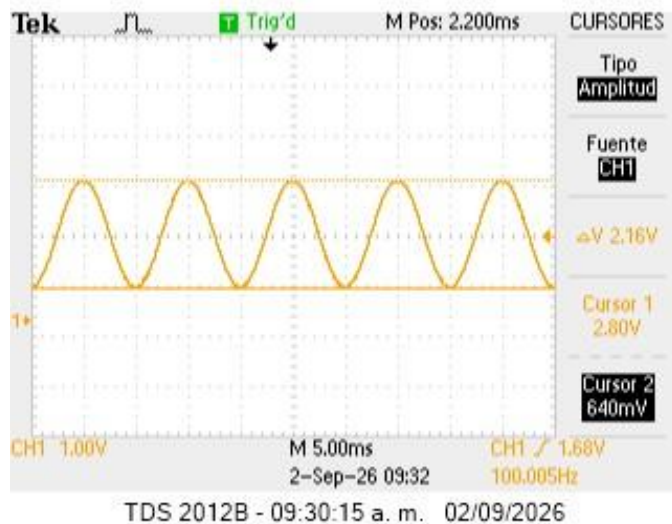


Figura 7. Verificación de la señal, antes de ingresarla a la Raspberry Pi Pico 2 W.

Como se observa en la Figura 7, el $V_{m\acute{a}x}$ tiene un valor de 2,16 V, mientras que el $V_{m\acute{i}n}$ corresponde a 640 mV. Estos valores permiten comprobar experimentalmente que la señal se encuentra dentro del rango establecido de 0 V a 3,3 V.

Por lo tanto, se confirma que la señal cumple con el rango permitido tanto teórico como experimentalmente, por lo que puede ser ingresada de manera segura a la Raspberry Pi Pico 2 W.

B. EJECUCION DE SAMPLING_1.PY (PARTE I)

Una vez verificado que la señal de entrada se encontraba dentro de los niveles de voltaje permitidos para la Raspberry Pi Pico 2W, se procedió a ejecutar el programa `sampling_1.py`, con el propósito de realizar la adquisición de la señal analógica mediante el convertidor analógico-digital (ADC) integrado en el microcontrolador. Para esta adquisición se utilizó el pin GP26 como entrada analógica y el pin GP15 como señal de marcación de los instantes de muestreo.

Al iniciar la ejecución, se verificó en la consola la configuración establecida por el programa, obteniéndose aproximadamente los siguientes parámetros:

- **ADC input:** GP26
- **Sampling marker:** GP15
- **Target Fs:** 1000 Hz
- **Target sampling time:** 1000 μ s

La frecuencia de muestreo configurada corresponde a un periodo de muestreo aproximado de 1000 μ s entre muestras. Durante la adquisición, la señal presente en GP15 fue observada mediante el osciloscopio en forma de un tren de pulsos digitales. Cada flanco ascendente de esta señal corresponde aproximadamente al instante inmediatamente anterior a una lectura del ADC, permitiendo identificar experimentalmente los instantes en los cuales se realiza la adquisición de la señal analógica.

Al finalizar la ejecución del programa se generó el archivo `datos.csv`, en el cual se almacenaron un total de 10.000 muestras. El archivo contiene las columnas `Sample`, `Time_us`, `Raw_u16` y `Voltage_V`, las cuales permiten identificar tanto la secuencia de adquisición como los valores obtenidos durante el proceso de conversión analógico-digital.

Sample	Time_us	Raw_u16	Voltage_V
0	186	17812	0,896919
1	1014	12691	0,639052
2	2009	13683	0,689004
3	3013	22117	1,113696
4	4010	34648	1,744692
5	5014	46651	2,3491
6	6010	53389	2,688391
7	7014	52444	2,640806
8	8011	44122	2,221753

Tabla 1. 8 primeros datos tomados del CSV generado

A partir de los datos registrados se verificó que el valor de `Time_us` presenta un incremento aproximado de 1000 μ s entre muestras consecutivas. Este comportamiento es consistente con la frecuencia de muestreo configurada de aproximadamente 1000 Hz, aunque pueden presentarse pequeñas variaciones debido al funcionamiento del sistema de adquisición y a la temporización del microcontrolador.

Para analizar gráficamente los resultados obtenidos, se utilizó Python para representar los datos almacenados en el archivo `datos.csv`. Se generaron dos gráficas principales: el voltaje estimado en función del tiempo y el valor digital `Raw_u16` en función del tiempo. Estas representaciones permiten analizar el comportamiento de la señal adquirida y observar cómo la variación de la señal analógica se refleja en las lecturas digitales proporcionadas por el ADC.

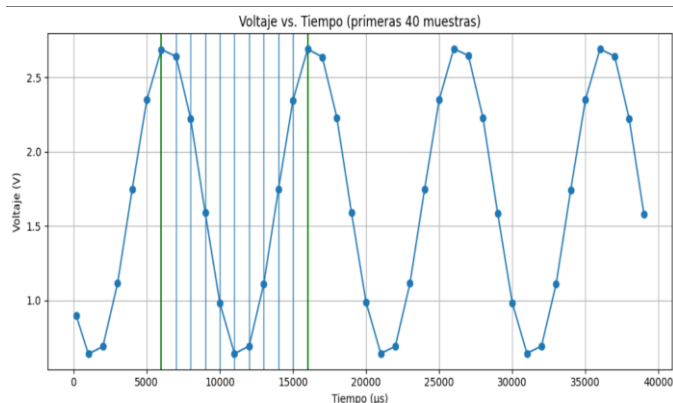


Figura 8. Gráfica de Voltaje en función del tiempo producida por Python.

En la gráfica de voltaje en función del tiempo se observa la variación de la señal adquirida a través del ADC. Los valores registrados se encuentran aproximadamente dentro del intervalo de 0,65 V a 2,65 V, correspondiente al rango de la señal previamente verificada antes de conectarla a la entrada GP26. Asimismo, la separación temporal entre las muestras permite evidenciar el proceso de muestreo realizado con una frecuencia cercana a 1000 Hz.

Considerando que la señal de entrada utilizada presenta una frecuencia de aproximadamente 100 Hz, una frecuencia de muestreo de 1000 Hz permite obtener aproximadamente 10 muestras por período. Esto proporciona una representación discreta de cada ciclo de la señal y permite observar su evolución temporal a partir de las muestras obtenidas.

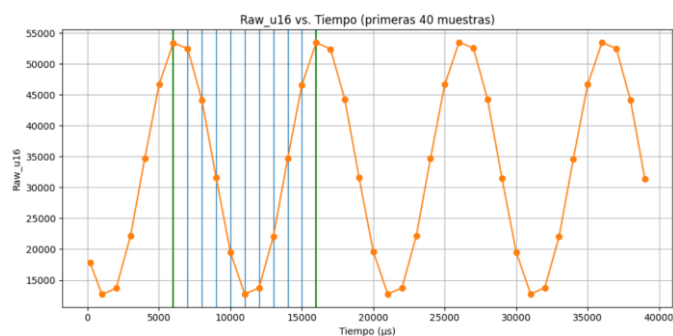


Figura 9. Gráfica de los valores digitales en función del tiempo producida por Python.

En la gráfica de Raw_u16 en función del tiempo se observa un comportamiento equivalente al de la señal de voltaje. Sin embargo, en este caso se representan directamente los valores digitales entregados por el ADC durante cada conversión. Por lo tanto, las variaciones de Raw_u16 corresponden a las variaciones de amplitud de la señal analógica aplicada a la entrada GP26.

La relación entre ambas representaciones permite comprobar que, a medida que aumenta o disminuye el voltaje de la señal

de entrada, también aumenta o disminuye el valor digital entregado por el ADC. De esta manera, se evidencia el proceso de conversión de la señal analógica a valores digitales realizado durante la adquisición.

Adicionalmente, en la representación gráfica se incluyeron líneas verticales para señalar de manera aproximada los instantes de muestreo determinados a partir de los datos registrados en el archivo CSV. Estas líneas permiten relacionar los instantes de adquisición calculados mediante Python con los flancos ascendentes de la señal GP15 observados experimentalmente en el osciloscopio.

Para complementar el análisis realizado mediante Python, se observó directamente en el osciloscopio la señal de entrada aplicada al pin GP26 junto con el tren de pulsos generado en GP15.

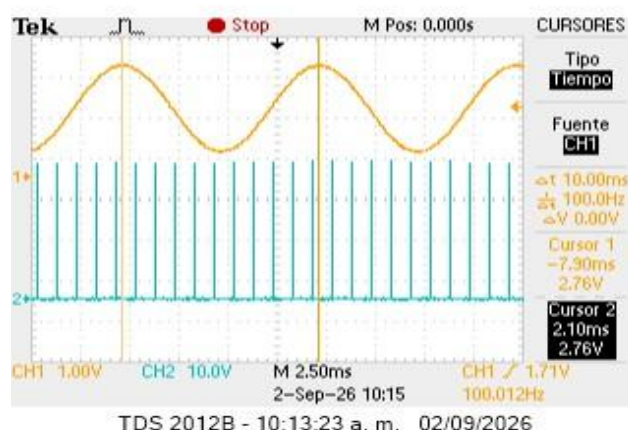


Figura 10. Señal de entrada en GP26 y tren de pulsos digitales generado por GP15 durante el proceso de muestreo.

En la Figura 10 se observa la señal analógica aplicada a la entrada GP26 y el tren de pulsos digitales correspondiente a la señal de marcación GP15. Los flancos ascendentes de GP15 permiten identificar aproximadamente los instantes inmediatamente anteriores a las lecturas realizadas por el ADC.

De esta manera, los resultados obtenidos mediante Python y las observaciones realizadas directamente en el osciloscopio permiten verificar experimentalmente el funcionamiento del proceso de muestreo implementado mediante `sampling_1.py`. Finalmente, las variables almacenadas en el archivo `datos.csv` se interpretan de la siguiente manera:

- **Número de la muestra:** corresponde a la columna `Sample`, que identifica cada muestra dentro de la secuencia de adquisición.
- **Instante de la muestra:** corresponde a la columna `Time_us`, que indica el instante en microsegundos (µs) en el que fue registrada cada muestra.

- **Lectura del ADC:** corresponde a la columna Raw_u16, que contiene el valor digital entregado por el ADC para cada muestra.
- **Voltaje estimado:** corresponde a la columna Voltage_V, que representa el voltaje estimado a partir de la lectura digital del ADC, expresado en voltios (V).

C. MEDICION EXPERIMENTAL DE LA FRECUENCIA DE MUESTREO.(PARTE I)

Como lo observamos en el apartado anterior, el programa tiene una frecuencia nominal, la cual es la siguiente:

$$F_s(nom) = 1000Hz$$

Por lo tanto el periodo nominal esperado es:

$$T_s(nom) = \frac{1}{1000Hz} = 1ms$$

Esto indica que cada pulso digital debe presentar una separación aproximada de 1 ms. Con base en estos valores, se tomó el tren de pulsos digitales generado por el GP15 para realizar la medición experimental de la frecuencia de muestreo mediante los cursores del osciloscopio.

Para realizar la medición, se utilizó CH2 y se amplió horizontalmente la señal hasta identificar claramente los flancos ascendentes de los pulsos. Posteriormente, se colocó el primer cursor sobre un flanco ascendente y el segundo cursor sobre el siguiente flanco ascendente, permitiendo determinar la diferencia de tiempo Δt entre ambos pulsos. Este procedimiento se repitió para varios pulsos consecutivos y los valores obtenidos fueron registrados en una tabla.

Como evidencia del procedimiento realizado, se presenta la siguiente captura del osciloscopio:



Figura 11. Procedimiento realizado con el osciloscopio

En la captura se observa una diferencia de tiempo entre los cursores de:

$$\Delta t = 1.000 ms$$

A partir de las mediciones realizadas, se obtuvieron los siguientes resultados:

Δt	$1/\Delta t$
1ms	1KHz
1,040ms	961,5Hz
1ms	1KHz
1ms	1KHz
1ms	1KHz
1ms	1KHz
1ms	1KHz
1ms	1KHz
1ms	1KHz
1ms	1KHz

Tabla 2. Datos obtenidos del osciloscopio para 10 muestras.

Los resultados muestran que 9 de las 10 mediciones registradas presentan un periodo de 1 ms, correspondiente a una frecuencia de 1 kHz, mientras que una medición presentó un periodo de 1,040 ms, equivalente a aproximadamente 961,5 Hz.

En base a esto podemos comprobar las mediciones experimentales de la siguiente manera:

Periodo de muestreo experimental:

$$T_s(osc) = \frac{\sum \Delta t}{\text{Numero de muestras}} = \frac{10,049ms}{10} = 1.004ms$$

Frecuencia de muestreo experimental:

$$T_s(osc) = \frac{1}{T_s(exp)} = \frac{1}{1.004ms} = 996,01Hz$$

Con los datos anteriores, completamos la siguiente tabla:

Magnitud	Resultado
F(nom)	1000Hz
Δt	1ms
T(osc)	996,01Hz
F(osc)	1,004ms

Tabla 3. Registro de Resultados

Hallamos el error:

$$e\% = \left| \frac{1000Hz - 996.01Hz}{1000Hz} \right| \times 100 = 0.399\%$$

La frecuencia obtenida en el osciloscopio fue de (996,01Hz), muy cercana a los (1000Hz) configurados en el programa. Con un error del (0,40%), esto confirma que el muestreo se realiza aproximadamente cada (1ms).

D. COMPARACION CON LA FRECUENCIA CALCULADA POR SOFTWARE (PARTE I)

El programa desarrollado permite obtener automáticamente la frecuencia de muestreo, el propósito de este apartado es comparar este resultado con la frecuencia de muestreo obtenida experimentalmente mediante las mediciones realizadas con el osciloscopio.

La frecuencia de muestreo entregada por el programa fue de:

$$1000.02\text{Hz}$$

Y la frecuencia de muestreo calculada a partir de las mediciones realizadas con el osciloscopio fue de:

$$996.01\text{Hz}$$

A partir de estos datos, completamos la siguiente tabla:

Fs(nom)	Fs(soft)	Fs(osc)
1000Hz	1000,02Hz	996,01Hz

Teniendo ya los datos registrados en la anterior tabla, calculamos el error respecto de la frecuencia programada:

Error del software respecto a 1000 Hz:

$$e\% = \left| \frac{1000\text{Hz} - 1000.02\text{Hz}}{1000\text{Hz}} \right| \times 100 = 0.002\%$$

Error del osciloscopio respecto a 1000 Hz ya calculado en el apartado C:

$$e\% = 0.399\%$$

Los resultados obtenidos son cercanos a la frecuencia nominal de 1000Hz. El software presentó un error menor 0,002% que el osciloscopio 0,399%, demostrando una mayor precisión en el valor entregado.

E. MEDICION DE LA FRECUENCIA DE LA SEÑAL ANALOGICA (PARTE I)

Se utilizó el canal CH1 del osciloscopio para realizar la medición experimental del período de la señal senoidal de entrada. Para obtener una medida precisa, se ubicaron los cursores verticales sobre dos puntos equivalentes de la señal, específicamente sobre dos cruces ascendentes consecutivos por el nivel de offset.

Como evidencia de la medición realizada, se presenta la siguiente captura del osciloscopio:

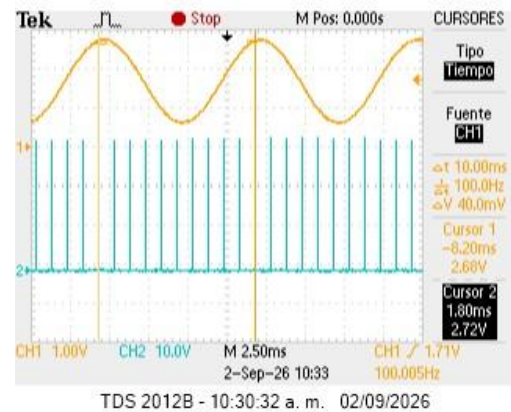


Figura 12. Medición de la señal analógica

A partir de la diferencia de tiempo entre ambos cursores, se obtuvo un período de:

$$T_{in}(osc) = 10\text{ms}$$

Con el período medido experimentalmente, se calculó la frecuencia de entrada mediante:

$$f_{in}(osc) = \frac{1}{T_{in}} = \frac{1}{10\text{ms}} = 100\text{Hz}$$

El valor obtenido mediante el osciloscopio fue de 100Hz, coincidiendo con el valor esperado de la señal.

F. NUMERO DE MUESTRAS POR PERIODO (PARTE I)

Como se ha establecido en los apartados anteriores, la señal presenta una frecuencia de aproximadamente 100Hz, mientras que la frecuencia de muestreo es de aproximadamente 1000Hz. Por lo tanto, se espera obtener aproximadamente 10 muestras por cada período de la señal. A partir de los valores obtenidos experimentalmente, se puede comprobar esta relación de la siguiente manera:

$$N_s = \frac{F_s(osc)}{f_{in}(osc)} = \frac{996.01\text{Hz}}{100\text{Hz}} = 9.96 \text{ muestras/T}$$

Con los valores nominales.

$$N_s = \frac{F_s(nom)}{f_{in}(nom)} = \frac{1000\text{Hz}}{100\text{Hz}} = 10 \text{ muestras/T}$$

Como vemos, los resultados son muy similares y comprueban que a 1000Hz como frecuencia de muestreo se generan 10 muestras por periodo, si calculamos el error podemos observar:

$$e\% = \left| \frac{10 \text{ muestras} - 9.96 \text{ muestras}}{10 \text{ muestras}} \right| \times 100 = 0.4\%$$

G. VERIFICACION VISUAL DE LAS MUESTRAS POR PERIODO (PARTE I)

Para comprobar el apartado F, verificamos cuantos pulsos digitales hay por cada periodo de la señal analógica, en la siguiente captura se evidencia lo generado por el osciloscopio:

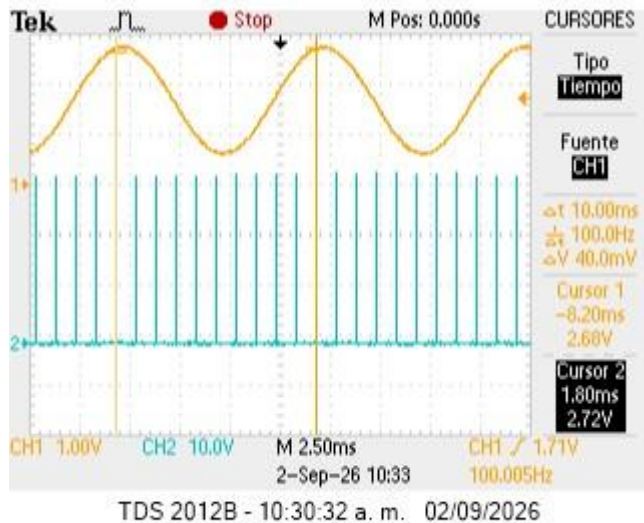


Figura 13. Numero de muestras por periodo de la señal analógica

Como vemos en la captura del osciloscopio, dentro de un periodo tenemos 10 muestras, lo que nos confirma que los cálculos realizados en el apartado F coinciden con la parte experimental.

H. EFECTO DE CAMBIAR LA FRECUENCIA DE LA SEÑAL (PARTE I)

Para esta parte del laboratorio se mantiene la frecuencia de muestreo de 1000Hz y la estructura del programa, lo único que cambia es el valor de la frecuencia del generador, para este caso se proponen los siguientes valores:

100 Hz, 200 Hz, 250 Hz, 400 Hz

Para cada caso la idea es calcular la cantidad de pulsos digitales por periodo y verificar que la frecuencia del generador coincida con la del osciloscopio, para que al final se pueda completar la siguiente tabla:

(f in) CONFIGURADA	(f in) OSCILOSCOPIO	Ns CALCULADO
100 Hz		
200 Hz		
250 Hz		
400 Hz		

Tabla 4. Registro de datos

1. f_{in} a 100Hz

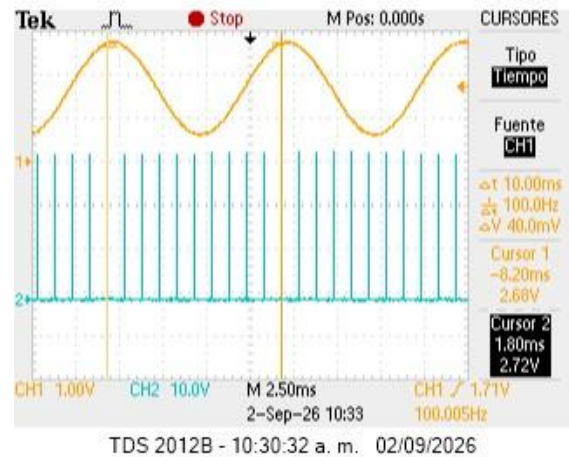


Figura 14. Primera frecuencia configurada

$$N_s = \frac{F_s(nom)}{f_{in}(nom)} = \frac{1000Hz}{100Hz} = 10 \text{ muestras}/T$$

Como observación, vemos que dentro de un periodo de la señal analógica obtenemos 10 pulsos digitales, lo que coincide con el cálculo inmediatamente anterior.

2. f_{in} a 200 Hz

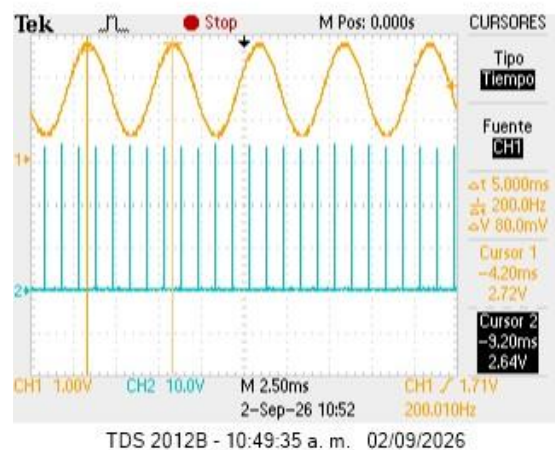


Figura 15. Segunda frecuencia configurada

$$N_s = \frac{F_s(nom)}{f_{in}(nom)} = \frac{1000Hz}{200Hz} = 5 \text{ muestras}/T$$

Como observación, vemos que dentro de un periodo de la señal analógica obtenemos 5 pulsos digitales, lo que coincide con el cálculo inmediatamente anterior.

3. f_{in} a 250 Hz

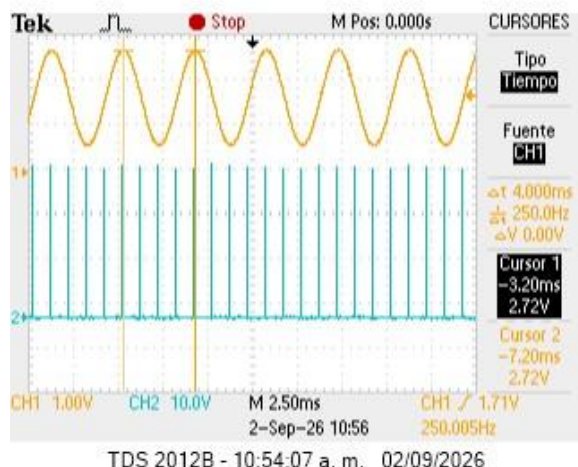


Figura 16. Tercera frecuencia configurada

$$N_s = \frac{F_s(nom)}{f_{in}(nom)} = \frac{1000Hz}{250Hz} = 4 \text{ muestras/T}$$

Como observación, vemos que dentro de un periodo de la señal analógica obtenemos 4 pulsos digitales, lo que coincide con el cálculo inmediatamente anterior.

4. f_{in} a 400 Hz

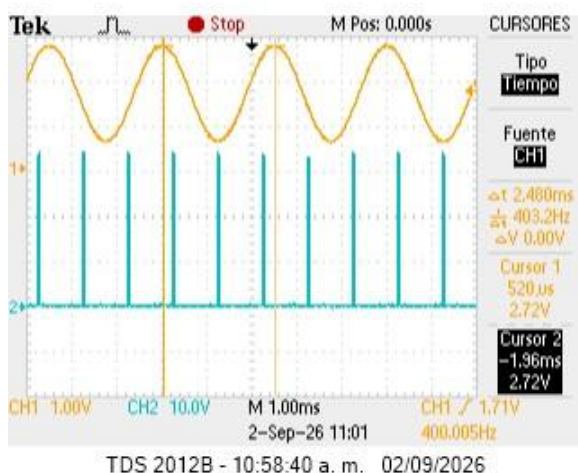


Figura 17. Cuarta señal configurada

$$N_s = \frac{F_s(nom)}{f_{in}(nom)} = \frac{1000Hz}{400Hz} = 2.5 \text{ muestras/T}$$

Como observación, vemos que dentro de un periodo de la señal analógica obtenemos 3 pulsos digitales aproximadamente, lo que coincide con el cálculo inmediatamente anterior.

Con los resultados anteriores, obtenemos como resultado:

(f_{in}) CONFIGURADA	(f_{in}) OSCILOSCOPIO	N_s CALCULADO
100 Hz	100Hz	10 muestras/T
200 Hz	200 Hz	5 muestras/T
250 Hz	250 Hz	4 muestras/T
400 Hz	400,005 Hz	2,5 muestras/T

Tabla 5. Resultados obtenidos de la practica

I. CONEXIÓN DEL SISTEMA Y EJECUCIÓN DE SAMPLING_2.PY (PARTE II)

Para la segunda parte de la práctica se conectó la entrada GP26/ADC0 de la Raspberry Pi Pico 2 W a una fuente de tensión DC, verificando previamente con un multímetro que la tensión aplicada se mantuviera dentro del rango permitido de 0 a 3,3 V. En esta práctica se caracterizaron estadísticamente cuatro niveles de tensión DC: 0,5 V, 1,5 V, 2,0 V y 2,5 V.

Para cada nivel de tensión se ejecutó el programa sampling_2.py, el cual realizó $N = 10\,000$ mediciones consecutivas del ADC y calculó en tiempo real, mediante el algoritmo de Welford, la media, la varianza muestral, la desviación estándar, los valores mínimo y máximo, y la distribución de los códigos nominales de 12 bits (code12 = raw16 >> 4). Al finalizar cada ensayo el programa generó dos archivos: samples_test_X.csv, con las 10 000 lecturas crudas (Raw_u16) obtenidas, e histogram_test_X.csv, con el número de ocurrencias de cada código nominal de 12 bits observado durante la adquisición.

En la Tabla siguiente se resumen los valores de media y desviación estándar reportados directamente por la consola de sampling_2.py para cada ensayo, junto con los archivos generados:

Tabla 6. Resultados de las mediciones realizadas con el ADC, reportados por sampling_2.py.

Test	VDDMM[V]	$\bar{V}_{ADC}$ [V] (firmware)	s [mV] (firmware)	Archivos generados
1	0,5	0,5611	5,577	samples_test_1.csv / histogram_test_1.csv
2	1,5	1,5208	5,953	samples_test_1.csv / histogram_test_1.csv
3	2,0	1,9758	5,722	samples_test_2.csv / histogram_test_2.csv
4	2,5	2,4264	5,920	samples_test_1.csv / histogram_test_1.csv

J. PROCESAMIENTO DE LOS DATOS EN MATLAB: SERIES TEMPORALES E HISTOGRAMAS (PARTE II)

Los archivos samples_test_X.csv generados en cada ensayo se descargaron y procesaron mediante el script de MATLAB analisis_ADC.m. Para cada nivel de tensión, el script convirtió las lecturas Raw_u16 a voltaje mediante $V_i = \text{Raw_u16} \cdot V_{REF}$

/ 65535, recuperó el código nominal de 12 bits mediante $\text{code12} = \text{raw16} \gg 4$, y calculó nuevamente la media $\bar{V}$, la desviación estándar s , el valor mínimo, el valor máximo y el rango de las mediciones. Adicionalmente, para cada ensayo se generaron: (i) una gráfica de las lecturas V_i en función del número de muestra, (ii) un histograma de las mediciones expresadas en voltios y (iii) un histograma de los códigos nominales de 12 bits.

Test 1 (0,5 V)

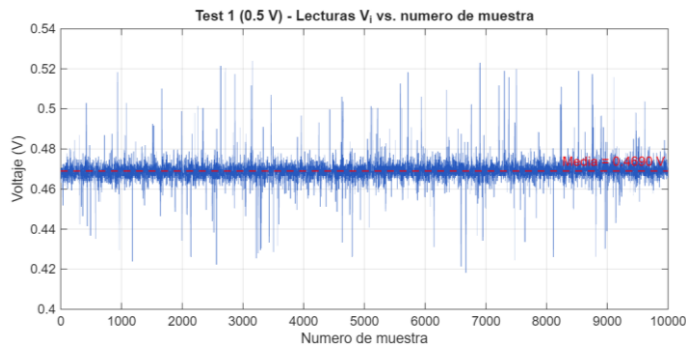


Figura 18. Test 1 (0,5 V) — lecturas V_i en función del número de muestra (media recalculada en MATLAB = 0,4690 V).

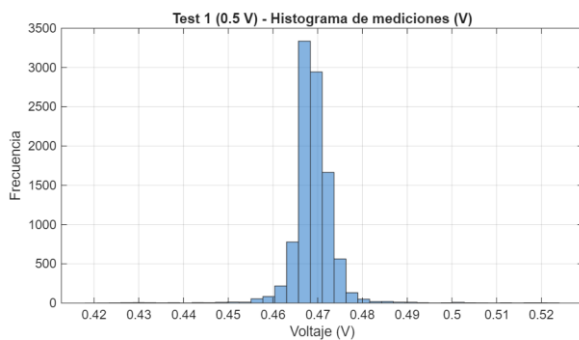


Figura 19. Test 1 (0,5 V) — histograma de las mediciones expresadas en voltios.

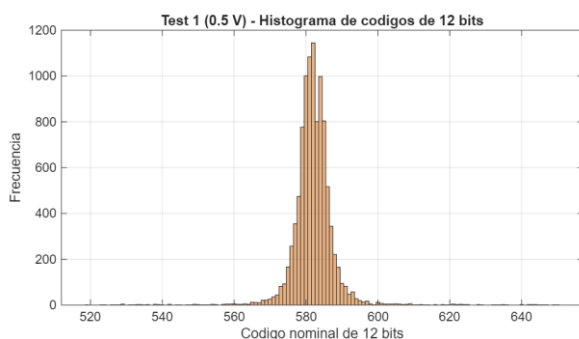


Figura 20. Test 1 (0,5 V) — histograma de los códigos nominales de 12 bits (111 códigos distintos observados).

Para este ensayo, la media recalculada en MATLAB fue de 0,4690 V, con una desviación estándar de 5,1145 mV, y se observaron 111 códigos de 12 bits distintos alrededor del valor central. Las gráficas muestran una serie temporal estable

alrededor de la media, con picos aislados producidos por ruido, y un histograma de forma aproximadamente acampanada (cuasi-gaussiana), tanto en voltios como en códigos digitales.

Test 2 (1,5 V)

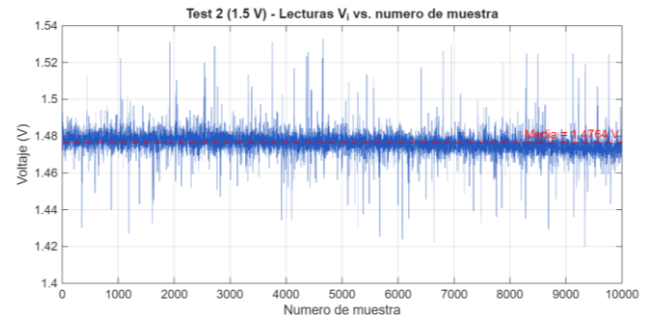


Figura 21. Test 2 (1,5 V) — lecturas V_i en función del número de muestra (media recalculada en MATLAB = 1,4764 V).

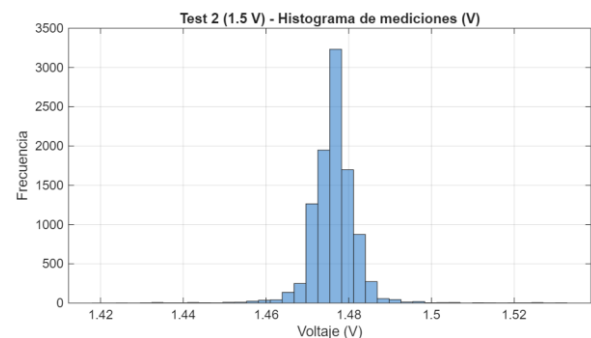


Figura 22. Test 2 (1,5 V) — histograma de las mediciones expresadas en voltios.

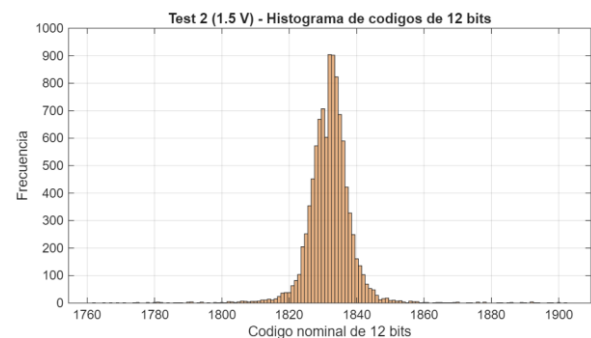


Figura 23. Test 2 (1,5 V) — histograma de los códigos nominales de 12 bits (109 códigos distintos observados).

Para este ensayo, la media recalculada en MATLAB fue de 1,4764 V, con una desviación estándar de 5,7850 mV, y se observaron 109 códigos de 12 bits distintos alrededor del valor central. Las gráficas muestran una serie temporal estable alrededor de la media, con picos aislados producidos por ruido, y un histograma de forma aproximadamente acampanada (cuasi-gaussiana), tanto en voltios como en códigos digitales.

Test 3 (2,0 V)

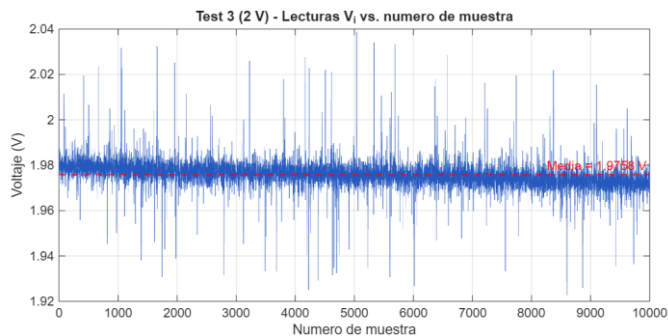


Figura 24. Test 3 (2,0 V) — lecturas V_i en función del número de muestra (media recalculada en MATLAB = 1,9758 V).

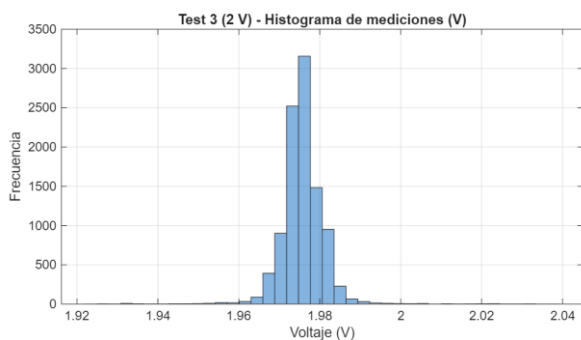


Figura 25. Test 3 (2,0 V) — histograma de las mediciones expresadas en voltios.

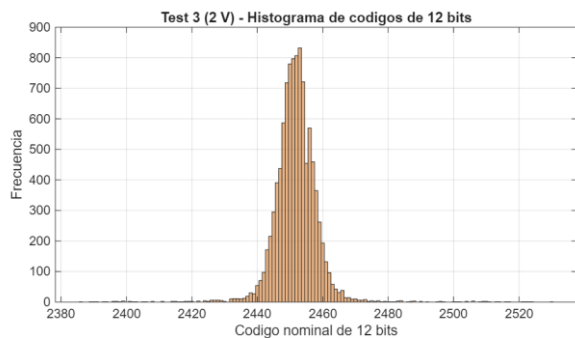


Figura 26. Test 3 (2,0 V) — histograma de los códigos nominales de 12 bits (111 códigos distintos observados).

Para este ensayo, la media recalculada en MATLAB fue de 1,9758 V, con una desviación estándar de 5,7208 mV, y se observaron 111 códigos de 12 bits distintos alrededor del valor central. Las gráficas muestran una serie temporal estable alrededor de la media, con picos aislados producidos por ruido, y un histograma de forma aproximadamente acampanada (cuasi-gaussiana), tanto en voltios como en códigos digitales.

Test 4 (2,5 V)

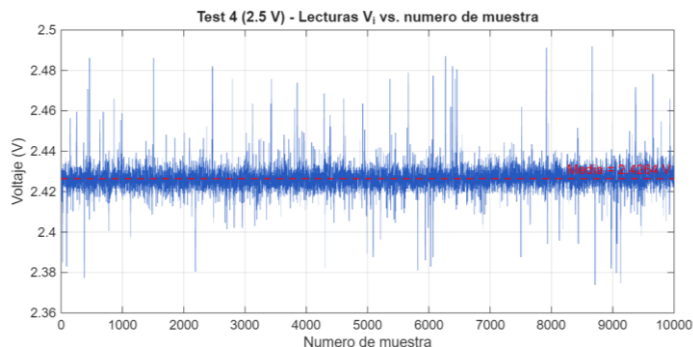


Figura 27. Test 4 (2,5 V) — lecturas V_i en función del número de muestra (media recalculada en MATLAB = 2,4264 V).

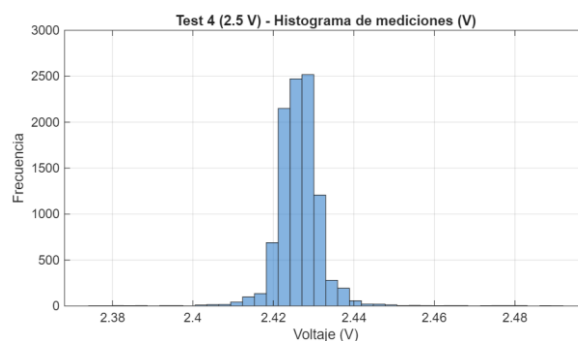


Figura 28. Test 4 (2,5 V) — histograma de las mediciones expresadas en voltios.

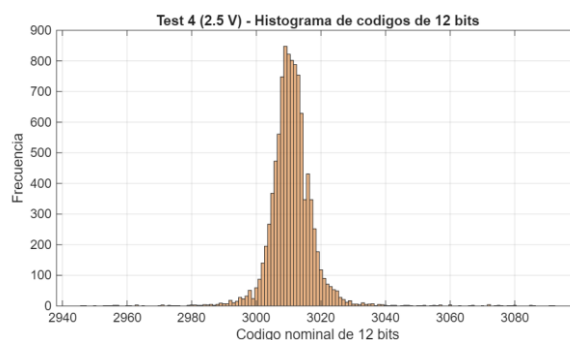


Figura 29. Test 4 (2,5 V) — histograma de los códigos nominales de 12 bits (110 códigos distintos observados).

Para este ensayo, la media recalculada en MATLAB fue de 2,4264 V, con una desviación estándar de 5,9208 mV, y se observaron 110 códigos de 12 bits distintos alrededor del valor central. Las gráficas muestran una serie temporal estable alrededor de la media, con picos aislados producidos por ruido, y un histograma de forma aproximadamente acampanada (cuasi-gaussiana), tanto en voltios como en códigos digitales.

K. COMPARACIÓN ENTRE LOS RESULTADOS DE MATLAB Y DEL FIRMWARE (PARTE II)

A continuación, se comparan los valores de media y desviación estándar recalculados en MATLAB a partir de los archivos

samples_test_X.csv, con los valores reportados originalmente por la consola de sampling_2.py durante la adquisición:

Tabla 7. Comparación entre los valores recalculados en MATLAB y los reportados por el firmware (sampling_2.py).

Test	$\bar{V}$ MATLAB [V]	$\bar{V}$ firmware [V]	$\Delta\bar{V}$ [mV]
1 (0,5 V)	0,469	0,5611	-92,08
2 (1,5 V)	1,4764	1,5208	-44,4
3 (2,0 V)	1,9758	1,9758	-0,01
4 (2,5 V)	2,4264	2,4264	-0,01

s MATLAB [mV]	s firmware [mV]	Δs [mV]
5,1145	5,577	-0,463
5,785	5,953	-0,168
5,7208	5,722	-0,001
5,9208	5,92	0,001

Para los ensayos 3 y 4 la media y la desviación estándar recalculadas en MATLAB coinciden prácticamente de forma exacta con los valores reportados por el firmware (diferencias menores a 0,01 mV en la media y a 0,002 mV en la desviación estándar), lo cual confirma que el cálculo en línea mediante el algoritmo de Welford (usado por sampling_2.py) y el cálculo por lotes realizado en MATLAB a partir del mismo archivo de muestras son equivalentes. En cambio, para los ensayos 1 y 2 se observa una diferencia apreciable entre ambos valores de media (-92,08 mV y -44,40 mV, respectivamente), y una diferencia menor en la desviación estándar. Dado que ambos cálculos parten exactamente del mismo archivo samples_test_X.csv, esta discrepancia no puede deberse al método de cálculo, sino muy probablemente a un error en la transcripción manual de los valores impresos en consola durante la adquisición de los ensayos 1 y 2 (por ejemplo, al registrar el resultado de una ejecución distinta a la que finalmente quedó guardada en el archivo CSV). Por esta razón, para el análisis estadístico siguiente se toman como válidos los valores recalculados en MATLAB directamente a partir de las muestras almacenadas.

L. ANÁLISIS ESTADÍSTICO DE LOS RESULTADOS (PARTE II)

Tomando como referencia los valores recalculados en MATLAB, se obtienen los siguientes parámetros derivados para cada ensayo (LSB ideal de 12 bits):

$$V_{REF} = 3,3V$$

$$LSB = \frac{3,3V}{4095} = 0,806mV$$

Tabla 8.. Parámetros estadísticos derivados de las mediciones del ADC para cada nivel de tensión DC.

Test	V _{DMM} [V]	$\bar{V}_{ADC}$ [V]	Error [mV]
1	0,5	0,469	-30,96
2	1,5	1,4764	-23,63
3	2	1,9758	-24,19
4	2,5	2,4264	-73,59

Error [%]	s [mV]	s [LSB]	Códigos 12 bits distintos
-6,19	5,1145	6,347	111
-1,58	5,785	7,179	109
-1,21	5,7208	7,099	111
-2,94	5,9208	7,347	110

a) Dispersión de las mediciones alrededor de la media.

La desviación estándar obtenida en los cuatro ensayos es pequeña frente al rango total del ADC (3,3 V), variando entre 5,11 mV y 5,92 mV, es decir, entre aproximadamente el 0,15 % y el 0,18 % del fondo de escala. Esto indica que, para una tensión DC aproximadamente constante, las 10 000 lecturas se agrupan de forma muy cerrada alrededor de su valor medio, con muy pocas muestras alejadas de este (visibles como picos aislados en las gráficas de serie temporal).

b) Forma de los histogramas obtenidos.

En los cuatro ensayos, tanto el histograma en voltios como el histograma de códigos de 12 bits presentan una forma unimodal, aproximadamente acampanada (cuasi-gaussiana), centrada en el valor medio de cada ensayo. Esta forma es característica de un proceso de medición afectado por ruido aleatorio (térmico y de conmutación) superpuesto a un valor DC constante, en lugar de un único código digital fijo como predeciría un ADC ideal sin ruido.

c) Cantidad de códigos ADC distintos para una tensión DC aproximadamente constante.

A pesar de aplicar una tensión prácticamente constante en cada ensayo, se observaron entre 109 y 111 códigos de 12 bits distintos por ensayo. Esto muestra que el ruido presente en el sistema de adquisición es considerablemente mayor que el tamaño de un único nivel de cuantización, por lo que una sola tensión DC termina activando más de un centenar de códigos digitales diferentes en lugar de uno solo, como ocurriría en un ADC ideal sin ruido.

d) Variación de la desviación estándar con el nivel de tensión.

En valor absoluto (mV), la desviación estándar aumenta ligeramente con el nivel de tensión aplicado, pasando de 5,11 mV en el ensayo de 0,5 V a 5,92 mV en el de 2,5 V. Sin embargo, al expresar la desviación estándar en unidades de LSB, los cuatro ensayos se mantienen en un rango estrecho

(6,35 a 7,35 *LSB*), lo que sugiere que la variabilidad observada está asociada principalmente al propio proceso de conversión y a las fuentes de ruido del sistema, y no depende de forma marcada del nivel de tensión de entrada.

e) Diferencia entre el valor promedio $\bar{V}_{ADC}$ y el valor medido con el multímetro V_{DMM} .

En los cuatro ensayos el valor promedio entregado por el ADC resultó sistemáticamente menor que el valor medido con el multímetro, con errores entre $-23,63 \text{ mV}$ y $-73,59 \text{ mV}$ (entre $-1,21 \%$ y $-6,19 \%$). Esta diferencia es notablemente mayor que la desviación estándar de cada ensayo (5 a 6 *mV*), lo que indica que no se trata de una fluctuación aleatoria, sino de un error sistemático (offset y/o error de ganancia) propio del ADC del RP2350 y de la referencia de voltaje utilizada.

f) Relación entre la desviación estándar obtenida y el LSB ideal de 12 bits.

El LSB ideal para un ADC de 12 bits con $V_{REF} = 3,3 \text{ V}$ es de aproximadamente 0,806 *mV*. Las desviaciones estándar medidas (5,11 a 5,92 *mV*) equivalen a entre 6,3 y 7,3 *LSB* ideales, es decir, son entre seis y siete veces mayores que el paso mínimo de cuantización teórico. Esto evidencia que el ruido real del sistema de adquisición domina sobre el error de cuantización ideal, y que la resolución efectiva del sistema es, en la práctica, inferior a los 12 bits nominales del convertidor.

g) Diferencia entre repetibilidad y exactitud observada experimentalmente.

Los resultados permiten distinguir claramente estos dos conceptos: la repetibilidad es buena, ya que las 10 000 mediciones de cada ensayo se agrupan de forma muy cerrada alrededor de su propia media (desviaciones estándar de apenas 5 a 6 *mV*). Sin embargo, la exactitud es pobre, pues ese valor medio se aleja de forma sistemática entre 24 *mV* y 74 *mV* del valor de referencia medido con el multímetro. Es decir, el ADC entrega resultados consistentes entre sí (repetibles), pero desplazados respecto al valor real de la tensión aplicada (poco exactos), lo cual es coherente con un error de *offset/ganancia* propio del convertidor más que con ruido aleatorio de medición.

VI. ANÁLISIS DE RESULTADOS

En conjunto, los resultados obtenidos en la Parte I muestran que el proceso de muestreo implementado mediante `sampling_1.py` se comporta de forma consistente con la teoría de muestreo de señales: la frecuencia de muestreo nominal de 1000 Hz fue reproducida con errores inferiores al 0,5% tanto por el software (0,002%) como por la medición directa con el osciloscopio (0,399%), lo que confirma que el reloj interno del RP2350 y la temporización del bucle de adquisición son suficientemente estables para esta aplicación. La leve diferencia entre el error del software y el del osciloscopio es atribuible a la resolución finita de los cursores del instrumento y no a una falla del sistema de adquisición. Asimismo, la variación controlada de la frecuencia de la señal de entrada (100, 200, 250 y 400 Hz) evidenció de forma consistente la relación inversa entre f_{in} y el número de muestras por período $N_s = \frac{F_s}{f_{in}}$ validando experimentalmente la expresión ($N_s = \frac{F_s}{f_{in}}$) y

mostrando visualmente en el osciloscopio la disminución progresiva de pulsos de muestreo por ciclo a medida que la señal se aproxima al límite impuesto por el teorema de muestreo.

En la Parte II, el análisis estadístico de las 10 000 mediciones por ensayo permitió separar dos fenómenos distintos que suelen confundirse en un sistema de adquisición real: la repetibilidad y la exactitud. Las desviaciones estándar obtenidas (entre 5,11 *mV* y 5,92 *mV*) resultaron entre 6 y 7 veces mayores que el LSB ideal de 12 bits (0,806 *mV*), lo que indica que el ruido eléctrico del sistema —y no la cuantización teórica— es el factor dominante en la dispersión de las lecturas, y que la resolución efectiva del ADC es, en la práctica, inferior a los 12 bits nominales. Por otro lado, el error sistemático entre el valor promedio del ADC y la lectura del multímetro (entre $-23,63 \text{ mV}$ y $-73,59 \text{ mV}$) fue notablemente mayor que la propia desviación estándar de cada ensayo, lo que descarta una fluctuación aleatoria y apunta a un error de offset y/o ganancia propio del conversor y de su referencia de voltaje interna.

La comparación entre los valores calculados en línea por `sampling_2.py` (algoritmo de Welford) y los recalculados por lotes en MATLAB confirmó la validez del cálculo estadístico del firmware: para los ensayos 3 y 4 ambos métodos coincidieron con diferencias menores a 0,01 *mV* en la media, mientras que las discrepancias observadas en los ensayos 1 y 2 se atribuyeron a un error de transcripción manual de los datos de consola y no a una falla del algoritmo, dado que ambos cálculos parten del mismo archivo de muestras. Finalmente, la relación funcional entre la resolución nominal de 12 bits del ADC, el escalamiento a 16 bits realizado por `read_u16()` y el tamaño del LSB permitió interpretar correctamente las 10 000 mediciones en términos físicos, evidenciando que, si bien la escala numérica cambia, la resolución física del convertidor —y por lo tanto su incertidumbre asociada— permanece determinada por los 12 bits reales del hardware.

VII. ANEXOS

Enlace

GitHub:

<https://github.com/Salomeblanc/Exploracion-de-conceptos-de-conversion-A-D-con-Raspberry-Pi-Pico-2-W/blob/main/README.md>

VIII CONCLUSIONES

Se verificó experimentalmente el proceso de conversión analógico-digital del ADC de la Raspberry Pi Pico 2 W: la frecuencia de muestreo configurada de 1000 Hz se reprodujo con un error de apenas 0,002% según el software y de 0,399% según las mediciones del osciloscopio, confirmando que el sistema de adquisición basado en `sampling_1.py` opera de forma confiable y predecible.

Se comprobó que el número de muestras por período ($N_s = \frac{F_s}{f_{in}}$) disminuye de forma inversamente proporcional a la frecuencia de la señal de entrada, pasando de 10 muestras/período a 100 Hz hasta aproximadamente 2,5 muestras/período a 400 Hz, resultado que coincidió tanto con los cálculos teóricos como con la observación directa en el osciloscopio.

El análisis estadístico de las 10 000 mediciones por nivel de tensión DC mostró desviaciones estándar de entre 5,11 mV y 5,92 mV, equivalentes a entre 6,3 y 7,3 LSB ideales, lo que demuestra que el ruido del sistema de adquisición domina sobre el error de cuantización teórico y que la resolución efectiva del ADC es inferior a sus 12 bits nominales.

Se estableció una diferencia clara entre repetibilidad y exactitud: el ADC entrega mediciones consistentes entre sí (desviaciones estándar de pocos milivoltios) pero sistemáticamente desplazadas respecto al valor de referencia medido con el multímetro (entre -23,63 mV y -73,59 mV), lo que evidencia un error de *offset/ganancia* propio del convertidor más que ruido aleatorio de medición.

Se relacionó de forma satisfactoria la resolución nominal de 12 bits del ADC con los 4096 niveles de cuantización, el tamaño ideal del LSB (0,806 mV para $V_{REF} = 3,3 V$) y el escalamiento realizado por la función `read_u16()` de MicroPython, confirmando que dicho escalamiento no aumenta la resolución física del conversor, sino que únicamente reexpresa el mismo código de 12 bits en una escala de 16 bits.

En general, la práctica permitió integrar los conceptos teóricos de muestreo, cuantización, resolución, repetibilidad y exactitud

con mediciones experimentales reales, evidenciando las diferencias que existen entre el comportamiento ideal de un ADC y el comportamiento real de un sistema de adquisición afectado por ruido y errores sistemáticos de hardware.

REFERENCIAS

- [1] Raspberry Pi Ltd, "Raspberry Pi Pico 2 W Datasheet," 2024. [En línea]. Disponible: <https://datasheets.raspberrypi.com/picow/pico-2-w-datasheet.pdf>
- [2] Raspberry Pi Ltd, "RP2350 Datasheet," 2024. [En línea]. Disponible: <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
- [3] MicroPython, "machine.ADC — analog to digital conversion," MicroPython Documentation. [En línea]. Disponible: <https://docs.micropython.org/en/latest/library/machine.ADC.html>
- [4] W. Kester, Ed., The Data Conversion Handbook. Burlington, MA: Newnes/Analog Devices, 2005.
- [5] A. V. Oppenheim y R. W. Schaffer, Discrete-Time Signal Processing, 3ra ed. Upper Saddle River, NJ: Pearson, 2010.
- [6] B. Sklar, Digital Communications: Fundamentals and Applications, 2da ed. Upper Saddle River, NJ: Prentice Hall, 2001.
- [7] Universidad Militar Nueva Granada, "Guía de laboratorio: Exploración de conceptos de conversión A/D con Raspberry Pi Pico 2W," Programa de Ingeniería en Telecomunicaciones.